

1. Contradictory documents. Retrieval finds `api-docs-v1.md` from 2021 saying token auth is supported, and `api-migration-guide-v2.md` from 2023 saying it's deprecated. Both go in the window. The model has no idea which one is current unless you tell it.

2. Bad chunking. This one is nasty:

Source document (sdk-errors.md)

Handling Rate Limit Errors

The SDK throws `RateLimitError` when you exceed 60 requests/min. Catch it and retry after waiting.

However, retrying immediately results in a permanent ban of your API key. Always wait at least 30 seconds before retrying.

chunk boundary lands here

Chunk A — "catch and retry after waiting"

similarity score: 0.91 → retrieved

Chunk B — "immediate retry = permanent ban"

similarity score: 0.60 → not retrieved

The model confidently tells you to retry immediately.

A chunk boundary in the wrong place doesn't produce a vague answer. It produces a confident answer that is exactly backwards.

3. Attention is biased — "lost in the middle." Even when the right document is in the window, position matters. Models attend strongly to the beginning and the end of a long context and weakly to the middle — the effect measured by Liu et al. (2023), *Lost in the Middle: How Language Models Use Long Contexts*.



Put the most important retrieved chunk near the top or the bottom of the prompt, not in the middle of forty others.

Context engineering

Which gives us three practical rules:

- **Curate.** Fewer, better, dated documents beat everything you own. Add source and last-modified metadata so the model can prefer the newer one.
- **Chunk well.** Respect semantic boundaries. Overlap chunks. Never split a warning away from the thing it's warning about.
- **Position for attention.** Most important material first or last.

RAG is soft conditioning, not a hard override. If your retrieved context says the newest model is "GPT 5.4" but the model's training says "GPT 3.5 Turbo", the context usually wins — but not always. Strongly memorized training facts can override what you put in the window. For anything that must be right, validate the output separately; don't assume retrieval settled it.